Part 18 — Questions for the Report

Local AI, Ollama, REST APIs, RAG, and Security

Note: Q13 requires your actual experimental measurements from Part 16. The answer has been left as a placeholder so that no results are invented. Q14 was not included in the supplied text.

Q1. What is Local AI?

Local AI is artificial intelligence that runs directly on a local computer instead of relying entirely on a remote server or cloud service. It allows users to run AI models on their own hardware.

Q2. Difference between a Cloud LLM and a Local LLM?

A Cloud LLM runs on remote servers and is accessed through the internet or an API. A Local LLM runs on the user's computer. Cloud models may offer more computing power, while local models provide greater control over data and can work without an internet connection after installation.

Q3. What role does Ollama play?

Ollama is a tool that allows users to download, manage, and run large language models locally. It simplifies model installation and provides an API that applications can use to send prompts and receive generated responses.

Q4. What is model quantization?

Model quantization is a technique that reduces the precision used to represent a model's weights, for example, from 16-bit to 4-bit values. This reduces memory consumption and can improve inference speed, although it may slightly reduce the model's accuracy or response quality.

Q5. Why does model size affect memory consumption?

A model contains parameters called weights that must be loaded into memory during inference. Larger models have more parameters and therefore require more memory. Additional memory is also needed for the context, intermediate calculations, and generated tokens.

Q6. Why can a smaller model sometimes be preferable?

A smaller model requires less RAM, can generate responses faster, and can run on computers with limited hardware resources. For simple tasks, it may provide sufficient quality without the additional cost and memory requirements of a larger model.

Q7. Advantages of local inference?

The main advantages of local inference are:

- **Privacy:** Data can remain on the local computer.
- **Offline access:** The model can run without an internet connection once installed.
- **Lower recurring costs:** There may be no per-request cloud API fees.
- **Control:** Users can choose and configure their models.
- **Reduced network dependency:** Inference does not depend on the availability of an external AI service.

Q8. Disadvantages of local inference?

Local inference requires sufficient RAM, storage, and processing power. It may be slower than cloud inference on computers with limited hardware. Smaller local models can also produce less accurate responses than larger models, and users are responsible for installation, updates, maintenance, and security.

Q9. Explain the architecture of your Python application.

My Python application uses a modular architecture. The Python backend receives requests from the user interface, processes the input, communicates with the local LLM through Ollama, and returns the generated response.

The main components are:

1. **User interface:** Allows users to enter prompts and view responses.
2. **Python backend:** Handles application logic, validates requests, and coordinates the different components.
3. **REST API:** Provides communication between the interface and the backend.
4. **Ollama:** Loads and runs the selected local language model.
5. **LLM:** Generates responses based on the provided prompts and context.
6. **Response handling:** Returns the generated text to the user interface.

If RAG is enabled, the application can also retrieve relevant information from documents and provide it to the model as additional context.

Q10. Explain the role of the REST API.

A REST API allows different parts of an application to communicate through HTTP requests. In my application, it allows the user interface to send prompts to the Python backend and receive the generated responses.

It separates the frontend from the backend, making the application easier to maintain, test, and integrate with other services.

Q11. What is RAG?

RAG stands for Retrieval-Augmented Generation. It is a technique that combines information retrieval with text generation.

First, the application searches a knowledge base or a collection of documents for relevant information. Then, it provides the retrieved information to the LLM as context for generating an answer.

RAG can improve the relevance of responses and allow a model to answer questions about specific documents without retraining the model.

Q12. Difference between an LLM and a multimodal LLM?

A traditional LLM primarily processes and generates text. A multimodal LLM can work with multiple types of data, such as text and images, and some models also support audio or video.

For example, a text-only LLM can answer a written question, while a multimodal LLM may analyze an image and answer questions about its contents.

Q13. Compare two local models experimentally.

Answer: Complete this section using the actual measurements and comparison table from Part 16. Include the model names, response time, memory use, and output quality or other metrics you measured. No experimental values were supplied in the prompt, so they have not been fabricated.

Q15. What security risks exist when exposing a local LLM server to a network?

Exposing a local LLM server creates several security risks, especially when the server has no authentication by default.

• **Unauthorized access:** Other users on the network may send requests to the server and use its resources without permission.

• **Resource abuse:** Attackers may submit excessive requests, consuming CPU, RAM, and other computing resources.

• **Model and data exposure:** If sensitive prompts, documents, or model files are accessible, unauthorized users may obtain confidential information.

• **Prompt injection:** Malicious instructions embedded in documents used by a RAG system may manipulate the model into ignoring intended instructions or revealing information available to it.

• **Denial of service:** A large number of requests or unusually long prompts may overload the computer.

For example, setting `OLLAMA_HOST=0.0.0.0` allows Ollama to listen on network interfaces rather than only on the local machine. This can make it accessible to other devices, depending on the network and firewall configuration. It does not automatically provide authentication or encryption.

To reduce these risks, I would:

1. Keep Ollama bound to localhost when remote access is unnecessary.
2. Use a firewall to restrict access to trusted devices.
3. If network access is required, place a reverse proxy in front of the service with authentication and HTTPS/TLS encryption.
4. Apply request limits and monitor access to prevent resource abuse.
5. Validate user input and treat retrieved documents as untrusted data to reduce prompt-injection risks.
6. Avoid exposing sensitive information through prompts, logs, or application responses.

In conclusion, running an LLM locally improves control over data, but it does not automatically make the application secure. Proper network configuration, access control, and input validation are essential when exposing the service to other devices.